Alice, as we noted in the previous section, is in the Project Managers group. This policy applies directly to her—it determines whether her request to access team reports is allowed based on the time and device she's using.

Instead of pushing this change directly into production, Carol treats it like any other software artifact. She commits the policy to version control, where it can be reviewed by a teammate, tested against known scenarios, and audited as part of ACME Corp's change management process. The PAP encompasses all this functionality.

Carol's work in the PAP transforms evolving business rules into secure, versioned policy logic that governs what users like Alice can do. This process supports agility without sacrificing control.

Once approved, the policy is deployed to production and used by the Policy Decision Point (PDP), which employs it to evaluate real-time authorization requests, such as the one Alice triggered earlier. Depending on the deployment model, the policy may take effect immediately in a centralized PDP or after a short delay in distributed systems where PDPs periodically pull updates or rely on cached policies.

At ACME, managing authorization policies like software artifacts—written in a declarative language, stored in version control, and reviewed through code workflows—has transformed how access is governed. Policies are no longer hidden inside application logic or buried in configuration files. They're externalized, testable, auditable, and traceable—making access control more reliable and adaptable to change.

4.5 Architecture patterns in practice

It's one thing to define components like the PEP, PDP, and PIP on a whiteboard. It's another to figure out where they actually reside in a working system. At ACME Corp, as in most real-world environments, the deployment of these components depends on factors such as architecture, performance, trust boundaries, and organizational structure.

Let's explore a few common patterns and how they come together in practice.